

레이다 SW 학습 과제 3

표적 궤적 모의

Target Trajectory Simulation

Janghwan Kim (Harley)

표적 궤적 모의 TargetSim

Janghwan Kim (Harley)

01

표적 구조체 설계

표적 하나를 어떤 숫자로 적을 것인가
기동을 어떻게 표현할 것인가

02

표적 궤적 모의 구현

동체 속도를 ECEF 속도로 바꾸는 방법
한 스텝을 전진시키는 방법

03

시뮬레이션 결과

명세 조건 60 초 결과와 검산
정확도 확인과 기동 시연

들어가며

과제가 요구한 것

「과제 3 내용」 문서의 요구사항을 네 덩어리로 묶고, 발표도 그 순서로 갑니다.

표적 구조체 정의

표적 최대 10 개, 표적마다 기동
최대 30 개.
위치는 위도 · 경도 · 고도,
속도는 동체
속도 (V_heading) 한 개로
입력받을 것.

표적 상태 갱신

위치와 속도는 ECEF 좌표계에서
갱신할 것.
추가 고민 — V_heading 을 ECEF
속도
Vx, Vy, Vz 로 바꾸는 방법.

시뮬레이션 조건

60 초 동안 0.1 초 간격.
플랫폼 1 + 대함 표적 1 + 대공
표적 1.
제공받은 좌표변환 코드를 그대로
사용.

결과 제시

궤적을 LLA 로 출력하고 그림으로
표현.
구조체 정의 · 구현 방법 · 결과를
코드와 함께 설명.

코드는 Part 2 에서 쓰던 좌표변환 라이브러리를 한 줄도 고치지 않고 그대로 가져다 썼습니다.
새로 만든 것은 TargetSim.c 하나 — 표적을 시간에 따라 움직이는 부분입니다.

표적을 왜 가짜로 만드나요 ?

레이다 소프트웨어를 만들려면 먼저 " 표적이 이렇게 움직였다 " 는 데이터가 있어야 합니다 .

같은 데이터를 얻는 두 가지 길

실제 해상 시험

- 함정과 항공기를 실제로 띄워야 한다
- 같은 상황을 두 번 만들 수 없다
- 날씨 · 공역 · 비용에 전부 묶인다
- 정답(참값)을 정확히 알 수 없다



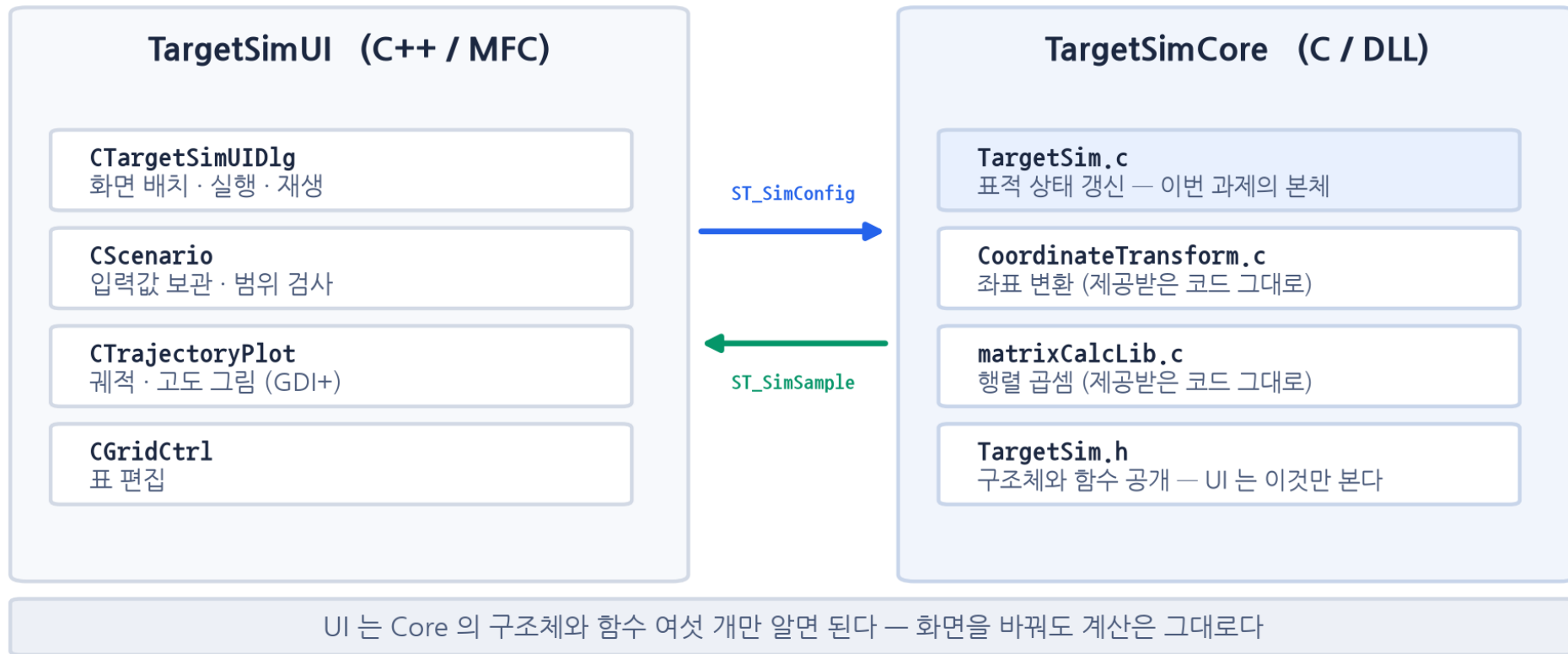
표적 모의 (이번 과제)

- 노트북 한 대에서 60초가 1초 만에
- 같은 시나리오를 몇 번이든 똑같이
- 표적 10개 · 기동 30번까지 자유롭게
- 내가 넣은 값이 곧 참값이다

데이터처리 소프트웨어가 받는 입력은 어느 쪽이든 똑같다
— 시각마다 찍힌 표적의 위도 · 경도 · 고도

만든 것 — 계산은 C, 화면은 MFC

계산 엔진과 화면을 나눠 두면 화면이 바뀌어도 계산은 건드릴 일이 없습니다.



01

표적 구조체 설계

Target Structure Design

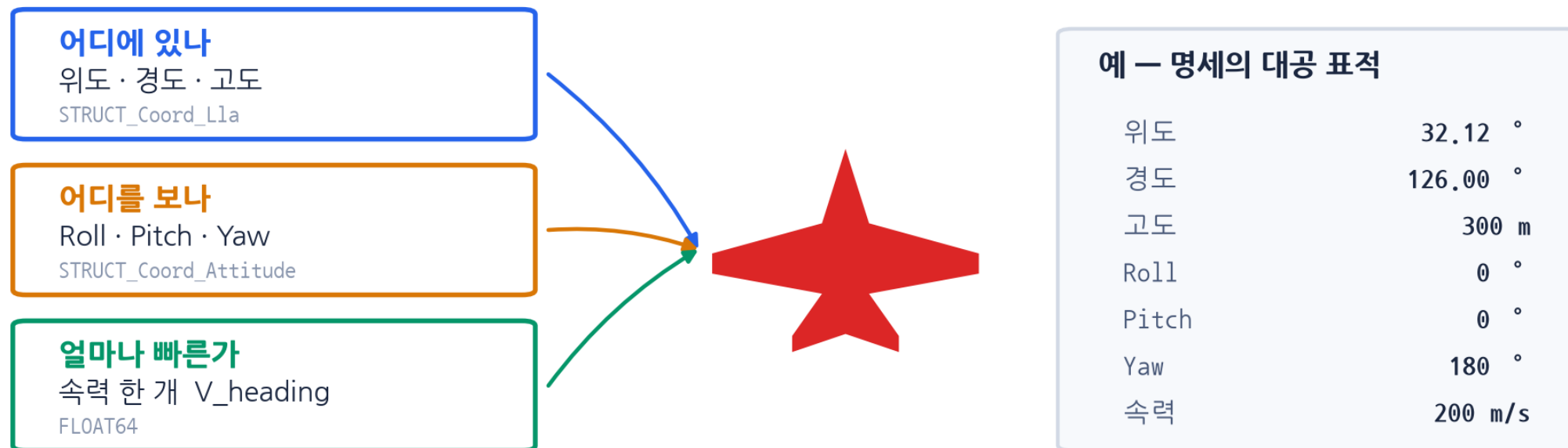
- 표적 하나를 숫자 몇 개로 적어야 충분한가
- "기동" 이라는 말을 코드가 알아들을 수 있는 값으로 바꾸기
- 설정 (입력) 과 상태 (출력) 를 나눈 이유

01. 표적 구조체 설계

표적 하나를 숫자로 적으면

지금 어디에 있고, 어디를 보고 있고, 얼마나 빠르냐 — 이 세 가지면 다음 순간을 계산할 수 있습니다.

표적 하나는 숫자 일곱 개로 적힌다

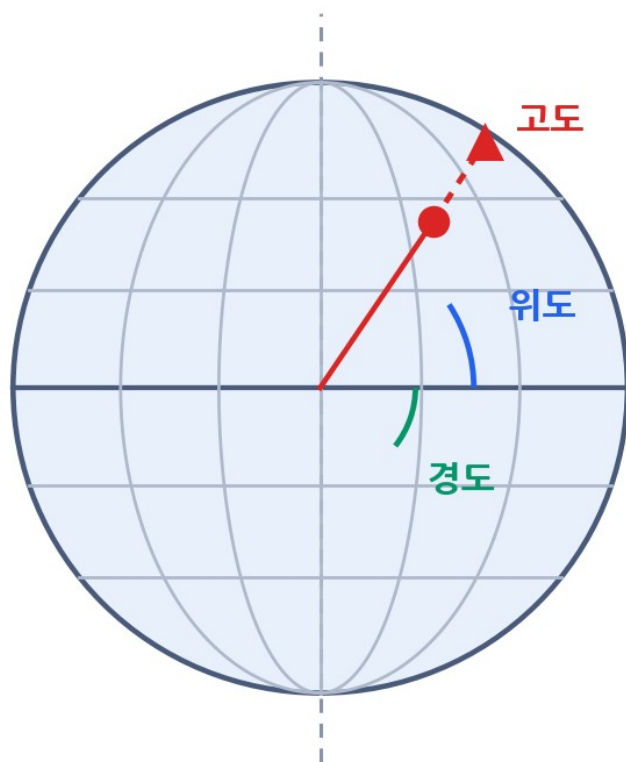


이 일곱 개가 표적 하나의 "초기 설정"이다. 여기에 "언제 어떻게 꺾을지"를 적은 기동표가 붙는다.

01. 표적 구조체 설계

위치는 위도 · 경도 · 고도로 받는다

명세가 정한 입력 형식이기도 하고, 사람이 지도에서 바로 찍어 볼 수 있는 값이기도 합니다.



위도 (Latitude)

적도에서 남북으로 잴 각

$-90^{\circ} \sim +90^{\circ}$

경도 (Longitude)

영국 그리니치에서 동서로 잴 각

$-180^{\circ} \sim +180^{\circ}$

고도 (Altitude)

지표면에서 위로 잴 높이

미터

사람이 읽기 좋은 좌표 — 그래서 입력과 출력은 LLA 로 받고 쓴다

01. 표적 구조체 설계

속도는 크기 하나만 받는다

방향은 자세각이 이미 들고 있으므로, 속도까지 방향을 갖게 하면 두 개가 서로 어긋납니다.

속도를 어떻게 입력받을 것인가

x, y, z 세 숫자로 받으면

"북서쪽으로 초속 30 m" 를
사람이 직접 세 숫자로 쪼개야 한다
세 숫자를 조금 잘못 넣으면
속력이 30 이 아니라 31.4 가 된다
자세각과 따로 놀아서, 기수는 서쪽인데
속도는 동쪽인 표적도 만들어진다

속력 하나 + 자세각으로 받으면

"기수 방향으로 초속 30 m" 그대로 적는다
조종사가 계기판에서 읽는 값과 같다
회전만 시키므로 속력은 절대 안 변한다
(회전은 길이를 바꾸지 않는다)
기동은 자세각만 돌리면 되고
속도 방향은 저절로 따라온다

명세도 "동체 속도(Heading 방향, V_heading) 성분으로 입력받도록" 을 요구한다

01. 표적 구조체 설계

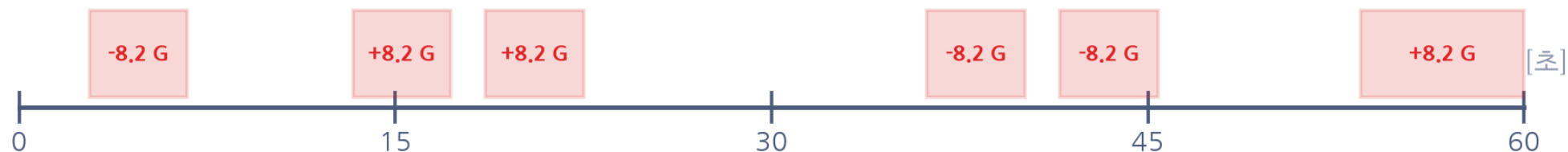
기동을 어떻게 적을 것인가

"12 초부터 16 초까지 오른쪽으로 8 G 로 꺾어라" — 이 한 문장을 네 개의 값으로 쪼갭니다.

기동 한 줄에 들어가는 것 네 가지

TurnType	Gravity Value	StartTime	EndTime
어느 축으로 0 없음 / 1 Roll 2 Yaw / 3 Pitch	얼마나 세게 중력가속도의 몇 배 부호가 회전 방향	언제부터 [초]	언제까지 [초]

표적 하나에 이런 줄을 최대 30개까지 — 걸린 구간에서만 기수가 돌아가고, 나머지 시간은 직진이다

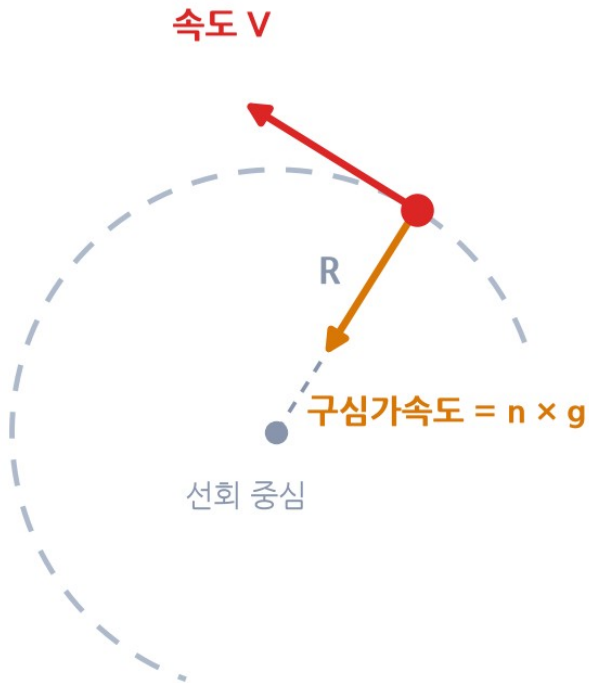


↑ 기동 시연 시나리오에서 대공 표적에 걸어 둔 여섯 줄

01. 표적 구조체 설계

G 값 하나가 회전 속도를 정한다

왜 각도 (°/s) 대신 G 로 받을까요? 조종사와 기체가 실제로 견디는 한계가 G 이기 때문입니다.



원운동은 식 한 줄이면 끝난다

$$V \times \omega = n \times g$$

속력 × 각속도 = 중력가속도의 n 배

$$\omega = (n \times 9.80665) \div V$$

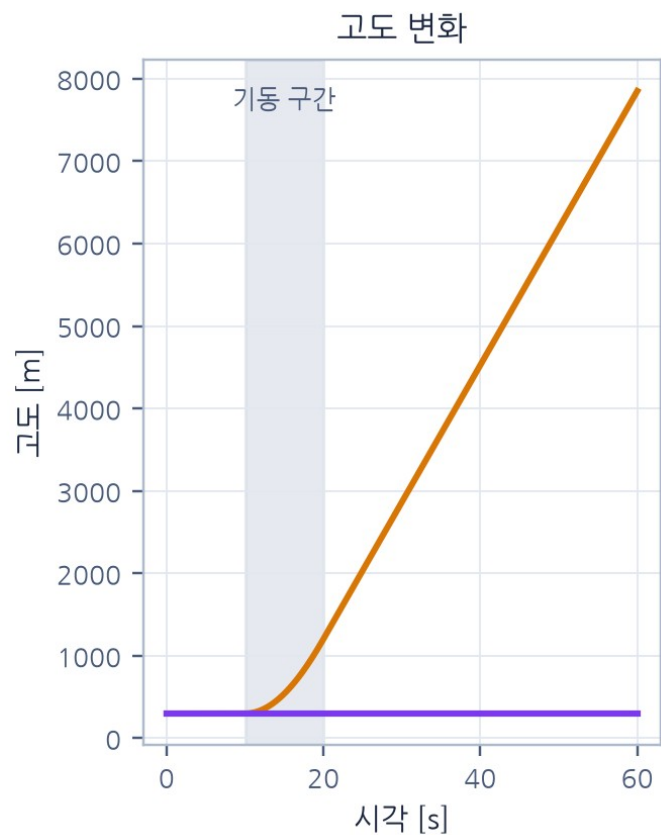
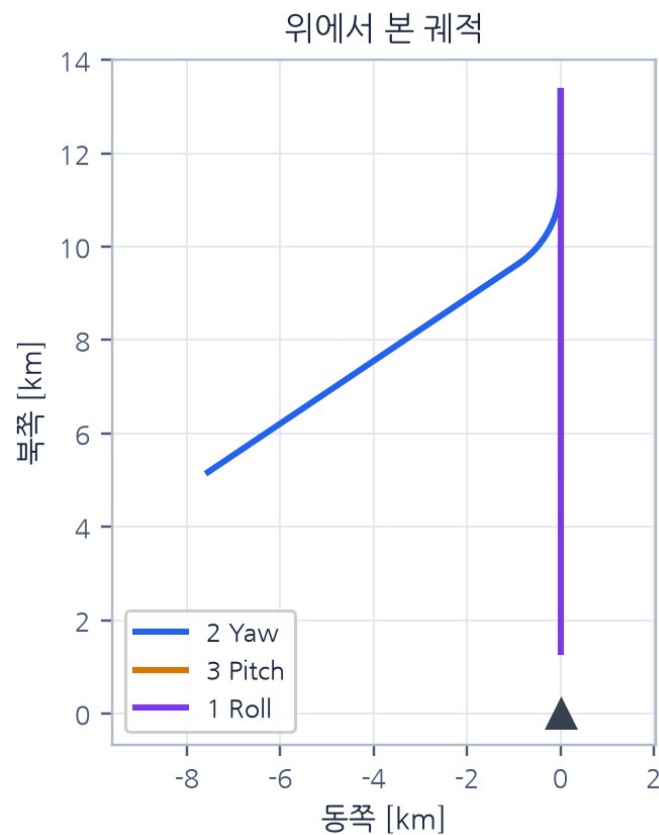
G 값 하나만 받으면 초당 몇 도를 돌지가 정해진다.

```
omega = (gravityValue * G_FORCE) / headingSpeed;
```

01. 표적 구조체 설계

회전축 세 가지 — 실제로 돌려 보면

같은 표적에 같은 2 G 를 10 초씩 , 축만 바꿔 걸어 본 결과입니다 .



같은 표적 · 같은 2 G · 같은 10초

Yaw

기수가 옆으로 돌아 궤적이 휘었다.
고도는 그대로.

Pitch

기수가 들려 7.5 km 를 올라갔다.
수평 방향은 거의 안 변함.

Roll

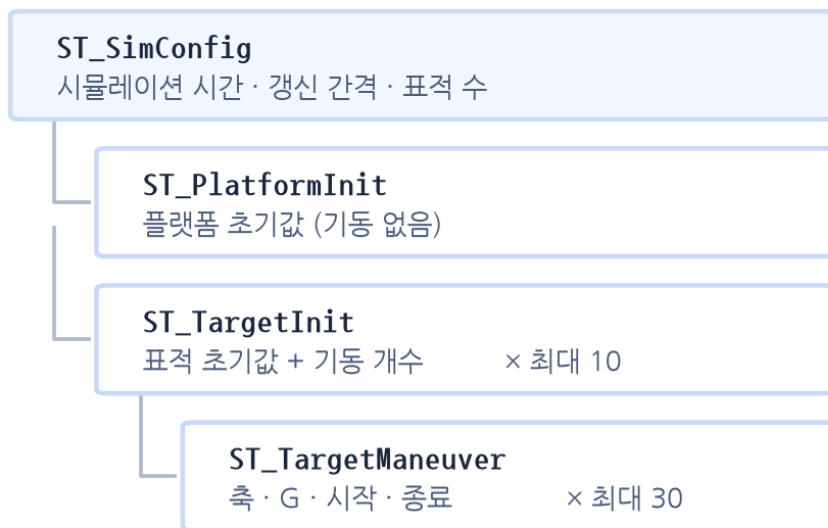
몸통만 돌았다. 속도 방향이 곧
회전축이라 궤적은 직진 그대로.

01. 표적 구조체 설계

구조체 전체 그림

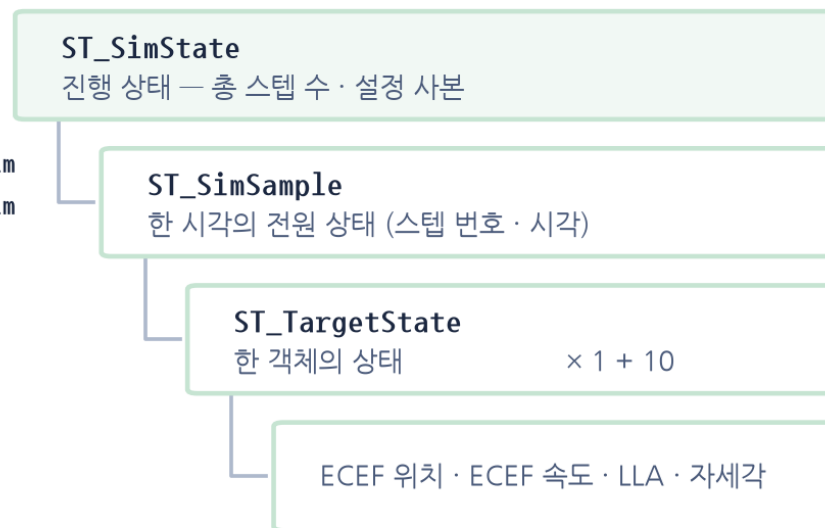
입력 쪽과 출력 쪽을 완전히 갈라 두었습니다.

입력 — 내가 적는 것



f_Tgt_InitSim
f_Tgt_StepSim

출력 — Core 가 채우는 것



설정과 상태를 따로 둔 이유 — 설정은 처음 한 번만 읽고, 매 스텝 갱신되는 것은 상태뿐이다

01. 표적 구조체 설계

코드로 보는 구조체 ① — 입력

TargetSimCore/include/TargetSim.h

```
// 기동 회전축 . 값은 과제 명세를 그대로 따른다 .
typedef enum
{
    TGT_TURN_NONE = 0,    TGT_TURN_ROLL  = 1,
    TGT_TURN_YAW  = 2,    TGT_TURN_PITCH = 3
} EN_TurnType;

// 기동 1 건 . gravityValue 의 부호가 회전 방향이다 .
typedef struct
{
    EN_TurnType    enTurnType;
    FLOAT64        gravityValue; // [G]
    FLOAT64        startTime;    // [s]
    FLOAT64        endTime;      // [s]
} ST_TargetManeuver;

typedef struct
{
    STRUCT_Coord_Lla    st_InitLla;
    STRUCT_Coord_Attitude st_InitAtt;
    FLOAT64             headingSpeed; // V_heading
    INT32               nManeuverNum;
    ST_TargetManeuver    st_Maneuver[TGT_MAX_MANEUVER_NUM];
} ST_TargetInit;
```

구간은 [시작, 종료)

한 기동의 끝과 다음 기동의 시작이 같아도 겹침이 아니라 이어 붙는 것으로 봅니다 .

겹치면 아예 거부

같은 표적에 같은 시각을 덮는 기동이 둘이면 TGT_ERR_MANEUVER_OVERLAP 을 돌려줍니다 .

10 × 30 은 배열로 고정

동적 할당을 쓰지 않아 실행 중 메모리 실패가 생길 자리가 없습니다 .

01. 표적 구조체 설계

코드로 보는 구조체 ② — 출력

설정은 처음 한 번만 읽고, 스텝마다 바뀌는 것은 상태뿐입니다.

```
// 한 시각의 상태 . 위치와 속도는 ECEF 에서 갱신하고 ,
// LLA 는 출력과 다음 스텝의 NED 기준으로 함께 보관한다 .
typedef struct
{
    FLOAT64          simTime;      // [s]
    STRUCT_Coord_Rect st_PosEcef;   // [m]
    STRUCT_Coord_Rect st_VelEcef;   // [m/s]
    STRUCT_Coord_Lla  st_Lla;
    STRUCT_Coord_Attitude st_Att;
} ST_TargetState;

// 한 시각의 플랫폼 · 표적 상태 .
typedef struct
{
    FLOAT64          simTime;
    INT32            nStepIndex;
    INT32            nTargetNum;
    ST_TargetState    st_Platform;
    ST_TargetState    st_Target[TGT_MAX_TARGET_NUM];
} ST_SimSample;
```

왜 ECEF 와 LLA 를 둘 다 들고 있나

계산은 ECEF 에서 하는 쪽이 정확하고,
출력과 화면은 LLA 라야 읽힙니다.
스텝마다 한 번만 바뀌 두면 쓸 때마다
다시 변환하지 않아도 됩니다.

플랫폼도 같은 구조체

플랫폼은 기동이 없는 표적일 뿐입니다.
같은 ST_TargetState 를 쓰고 각속도만
0 으로 넣어 같은 경로로 전진시킵니다.

02

표적 궤적 모의 구현

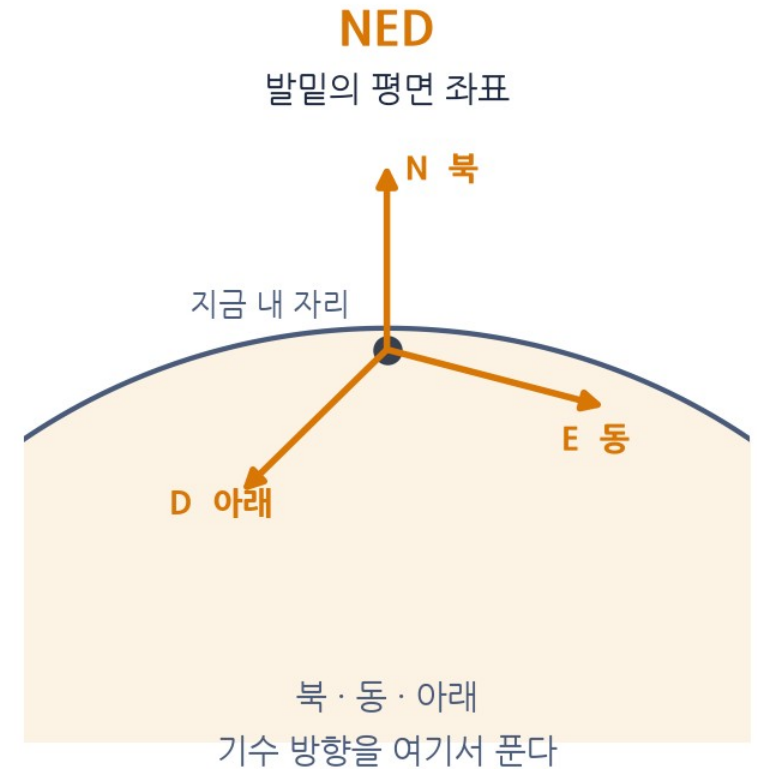
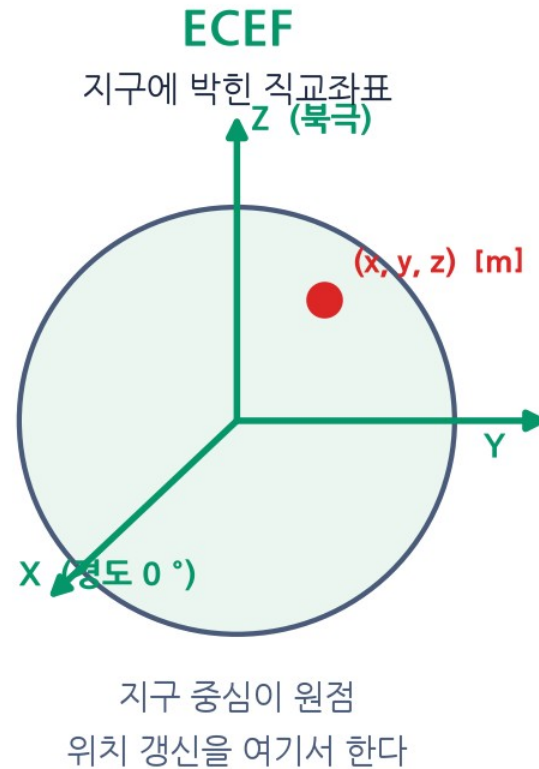
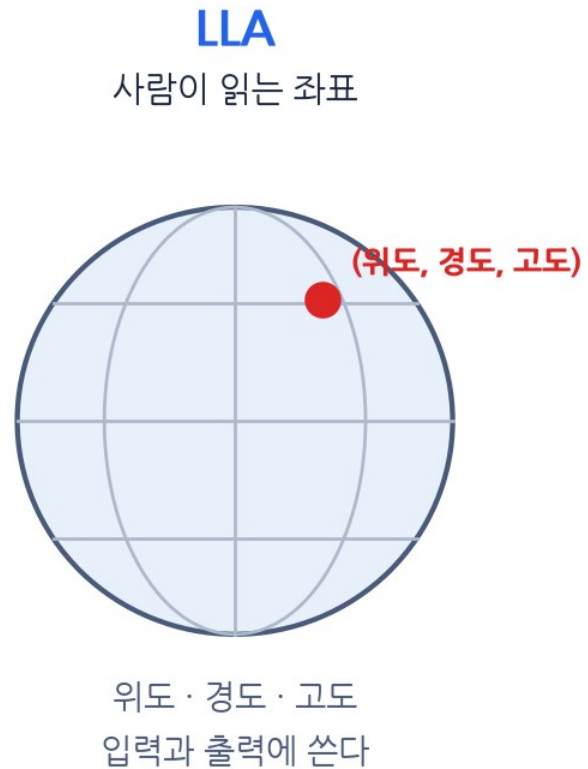
Trajectory Propagation

- 기수 방향 속력 하나를 ECEF 속도 세 성분으로 바꾸기
- 0.1 초를 한 번 전진시키는 방법 — 중점법
- 숫자가 깨졌을 때 상태를 지키는 방법

02. 표적 궤적 모의 구현

쓰는 좌표계는 셋뿐입니다

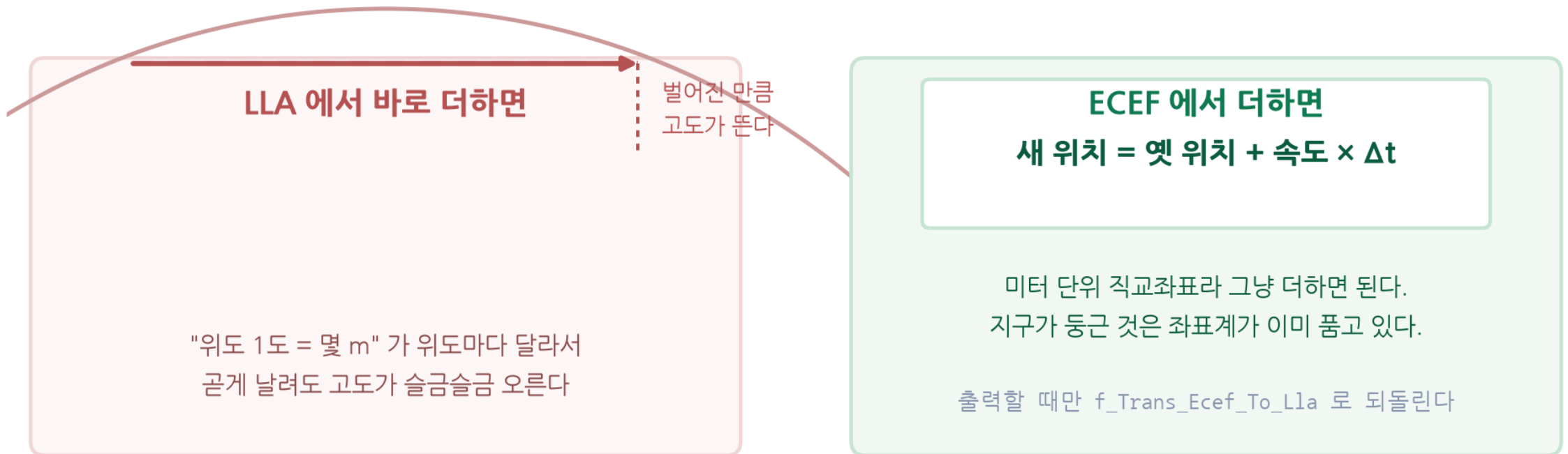
Part 2 에서 여섯 개를 봤지만, 표적을 움직이는 데 실제로 필요한 것은 이 셋입니다.



02. 표적 궤적 모의 구현

왜 ECEF 에서 갱신하는가

명세가 정한 조건이기도 하지만, 실제로 계산해 보면 이유가 분명합니다.



02. 표적 궤적 모의 구현

동체 속도를 ECEF 속도로 — 과제의 추가 고민

" 표적 동체 속도 값 (V_{heading}) 을 ECEF 좌표계 V_x, V_y, V_z 성분으로 변환하는 방법 " 에 대한 답입니다.

회전 두 번 — 그래서 속력은 처음 그대로 보존된다



실제로 60초 내내 재보면 — 대함 30.000000000 m/s, 대공 200.000000000 m/s
소수점 아홉 자리까지 넣은 값 그대로였다.

02. 표적 궤적 모의 구현

코드 ③ — 두 줄이면 끝난다

TargetSimCore/src/TargetSim.c

```
// 동체 속도 (V, 0, 0) 을 자세로 한 번 , 위경도로 한 번 돌려 ECEF 속도로 만든다 .
// 회전만 거치므로 속력은 그대로 보존된다 .
static STRUCT_Coord_Rect f_Tgt_GetVelEcef(const STRUCT_Coord_Attitude *st_Att,
    const STRUCT_Coord_Lla *st_Lla, FLOAT64 headingSpeed)
{
    STRUCT_Coord_Rect st_VelNed;

    // 참고 소스는 인자를 Roll, Yaw, Pitch 순서로 받는다 .
    st_VelNed = f_Trans_Body_To_Ned(headingSpeed, 0.0, 0.0,
        st_Att->Roll, st_Att->Yaw, st_Att->Pitch);

    // 방향만 옮길 때는 기준점을 원점으로 준다 .
    return f_Trans_Ned_To_Ecef(st_VelNed.x, st_VelNed.y, st_VelNed.z,
        0.0, 0.0, 0.0, st_Lla->Lat, st_Lla->Lon);
}
```

왜 기준점이 0 인가

f_Trans_Ned_To_Ecef 는 회전에 더해
기준점만큼 평행이동까지 합니다 . 속도는
" 위치 " 가 아니라 " 방향 " 이라 빼야 합니다 .

인자 순서를 왜 적어 두었나

제공받은 함수가 Roll, Yaw, Pitch 순서로
받습니다 . 자세각 구조체의 필드 순서와
달라 , 헷갈리는 자리라 주석을 남겼습니다 .

속력 보존 확인

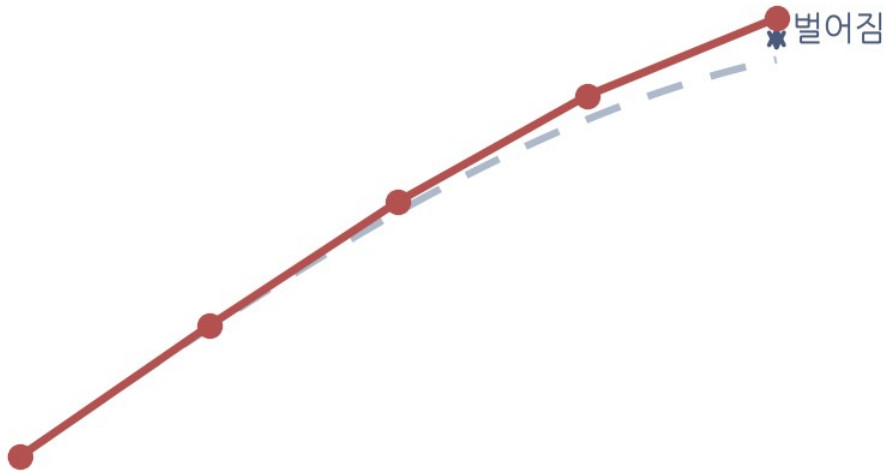
회전행렬은 길이를 바꾸지 않습니다 .
60 초 601 개 표본에서 속력을 재보면
200.000000000 m/s 가 601 번 나옵니다 .

02. 표적 궤적 모의 구현

0.1 초를 어떻게 전진시킬 것인가

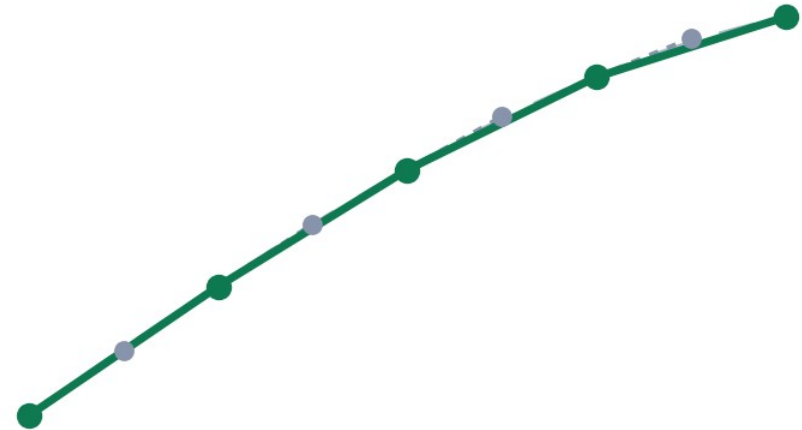
속도가 계속 바뀌는데 한 방향으로만 밀면 안쪽 곡선을 못 따라갑니다.

오일러법 — 출발할 때의 방향으로 쭉



굽은 길을 직선 네 토막으로 근사 — 바깥쪽으로 밀린다

중점법 — 반 걸음 가보고, 거기 방향으로 한 걸음



같은 0.1초인데 오차는 스텝 간격의 제곱으로 줄어든다

02. 표적 궤적 모의 구현

코드 ④ — 한 스텝 전진

TargetSimCore/src/TargetSim.c f_Tgt_Propagate (발췌)

```
halfStep = 0.5 * stepTime;

// ① 반 스텝 뒤의 자세
st_AttMid.Roll = st_State->st_Att.Roll + (st_Rate->Roll * halfStep);
st_AttMid.Pitch = st_State->st_Att.Pitch + (st_Rate->Pitch * halfStep);
st_AttMid.Yaw = st_State->st_Att.Yaw + (st_Rate->Yaw * halfStep);

// ② 반 스텝 뒤의 위치 — 스텝 시작 속도로 예측
st_VelStart = f_Tgt_GetVelEcef(&st_State->st_Att, &st_State->st_Lla, headingSpeed);
st_PosMid.x = st_State->st_PosEcef.x + (st_VelStart.x * halfStep);
...

// ③ 그 자리 · 그 자세에서 구한 속도가 이 스텝의 대표 속도
st_LlaMid = f_Trans_Ecef_To_Lla(st_PosMid.x, st_PosMid.y, st_PosMid.z);
st_VelMid = f_Tgt_GetVelEcef(&st_AttMid, &st_LlaMid, headingSpeed);

// ④ 그 속도로 한 스텝을 옮긴다
st_Next.st_PosEcef.x = st_State->st_PosEcef.x + (st_VelMid.x * stepTime);
...
```

왜 반 스텝 위치까지 다시 구하나

자세만 반 스텝 돌리고 위치는 그대로 두면,
지구가 둥근 만큼 방향이 뒤흔칩니다.
그러면 수평으로 날아도 고도가 매 스텝
 $d^2/2R$ 씩 올라갑니다.

짐벌락은 알고 남겨 두었다

오일러각을 직접 적분하므로 Pitch 가
 $\pm 90^\circ$ 근처면 Yaw 축이 속도 방향과 겹쳐
Yaw 기동이 궤적을 바꾸지 못합니다.
코드에 주석으로 명시해 두었습니다.

02. 표적 궤적 모의 구현

한 스텝에 벌어지는 일 전부

그리고 계산이 깨졌을 때 상태를 지키는 방법.

f_Tgt_StepSim 한 번 = 플랫폼 1 개 + 표적 N 개에 대해 아래를 반복



여섯 단계 어느 하나라도 숫자가 무한대나 NaN 이 되면 — 상태를 건드리지 않고 오류 코드만 돌려준다
(사본에서 전부 계산한 뒤, 다 성공했을 때만 한 번에 반영)

03

시뮬레이션 결과

Simulation Results

- 명세 조건 그대로 60 초를 돌린 결과
- 손으로 계산한 값과 맞는지 숫자로 검산
- 갱신 간격을 바꿔 가며 확인한 정확도

03. 시뮬레이션 결과

시뮬레이션 조건

「과제 3 내용」 2 항의 값을 그대로 넣었습니다.

	위도 [°]	경도 [°]	고도 [m]	속력 [m/s]	Roll [°]	Pitch [°]	Yaw [°]
플랫폼	32.0	126.0	0	0	0	0	0
대함 표적	32.125	126.03	0	30	0	0	270
대공 표적	32.12	126.0	300	200	0	0	180

시뮬레이션 시간 60 초

갱신 간격 0.1 초 → 601 개 표본
($t = 0$ 과 $t = 60$ 을 모두 포함)

Yaw 270° 는 서쪽

Yaw 는 북쪽이 0, 시계 방향이 +.
대함은 서쪽, 대공은 남쪽 (180°) 으로 갑니다.

기동 없는 등속 직진

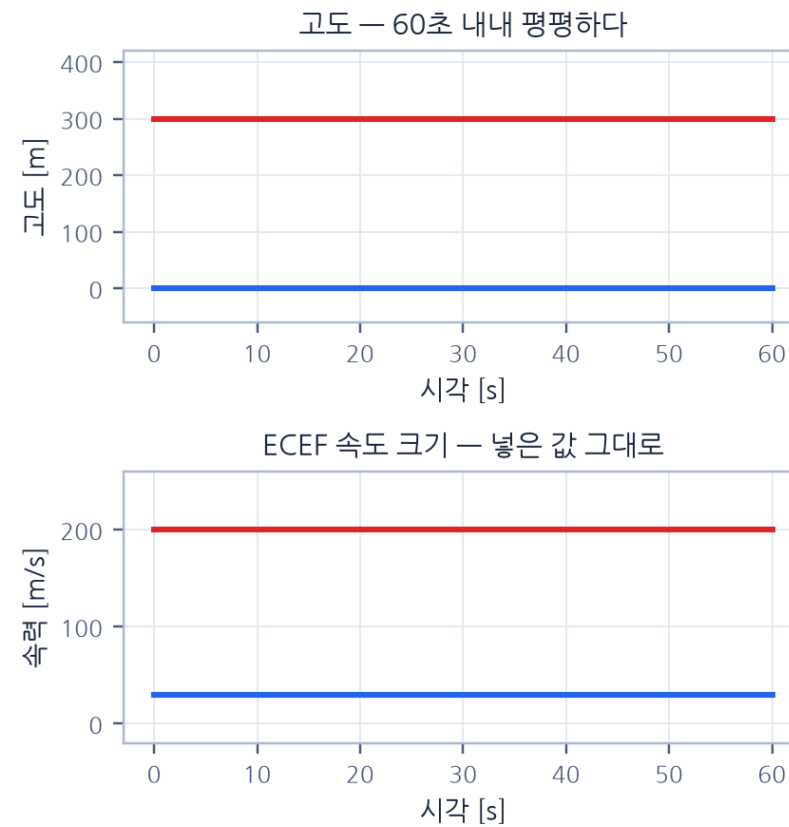
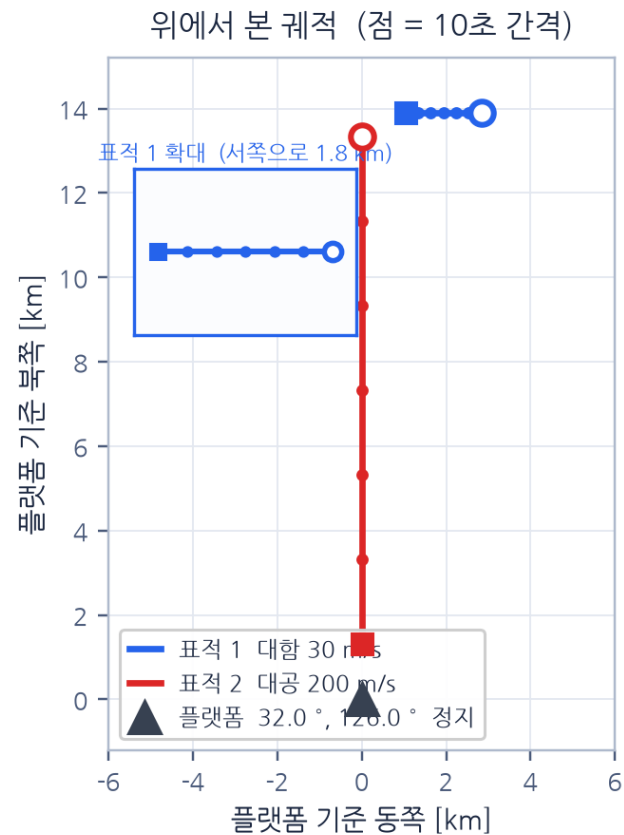
먼저 기동 없이 돌려 손계산과 맞춰 보고,
그 다음 기동을 엮었습니다.

플랫폼 속력이 0 이라 $f_{Tgt_Propagate}$ 의 나눗셈 분모가 되는 자리가 없는지 따로 확인했습니다 — 각속도 계산은 기동이 있을 때만 들어갑니다.

03. 시뮬레이션 결과

결과 ① — 60 초 궤적

플랫폼을 원점으로 잡은 동 - 북 평면입니다. 점 하나가 10 초입니다.



03. 시뮬레이션 결과

결과 ② — 손으로 계산해 보면

속력 × 시간, 고도 유지, 위도 유지 — 답을 미리 아는 항목으로 맞춰 봤습니다.

검산 항목	이론값 (손으로 계산)	시뮬레이션 결과	차이
대함 표적이 60초 동안 간 거리	$30 \text{ m/s} \times 60 \text{ s} = 1,800 \text{ m}$	1799.99996 m	-0.00004 m
대공 표적이 60초 동안 간 거리	$200 \text{ m/s} \times 60 \text{ s} = 12,000 \text{ m}$	11999.99995 m	-0.00005 m
대함 고도 (수평 서진)	0 m 유지	0.0000 ~ 0.0000 m	0
대공 고도 (수평 남진)	300 m 유지	300.0000 ~ 300.0000 m	0
대함 위도 (정서쪽으로만)	32.125 ° 유지	32.125000000 ° 고정	0
대공 경도 (정남쪽으로만)	126.0 ° 유지	126.000000000 ° 고정	0

경로 길이는 0.1초마다 찍힌 601개 점 사이 거리를 전부 더해서 낸 값이다.
12 km 를 날아 오차가 0.05 mm 수준이면, 남은 오차는 계산 방법이 아니라 double 형이 표현할 수 있는 한계에 가깝다.

03. 시뮬레이션 결과

결과 ③ — 0.1 초는 충분한 간격인가

직진은 틀릴 수가 없으니, 가장 많이 휘는 선회 구간으로 확인했습니다.



읽는 법

두 선이 나란하다

간격을 절반으로 줄이면 오차가 1/4로 준다.
중점법이 제대로 2차 정확도로 돌고 있다는 뜻.

0.1초에서 1.4 cm

20초를 2 G로 꺾는 동안 쌓인 오차가 1.4 cm.
레이다 거리 분해능(수 m)에 비하면 없는 값이다.

직진이면 더 작다

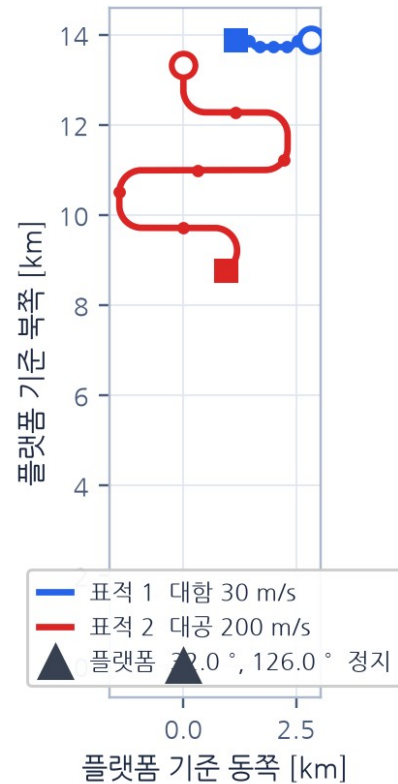
기동이 없을 때는 오차가 10 나노미터 수준 —
사실상 계산 오차만 남는다.

03. 시뮬레이션 결과

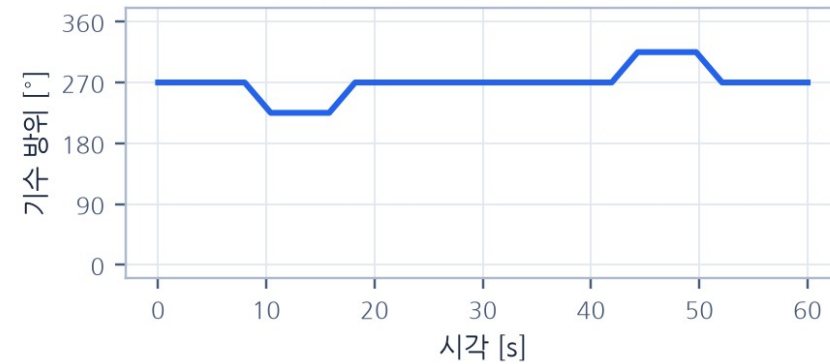
결과 ④ — 기동을 없으면

같은 초기값에 기동 10 줄만 추가했습니다. 초기값 · 시간 · 간격은 명세 그대로입니다.

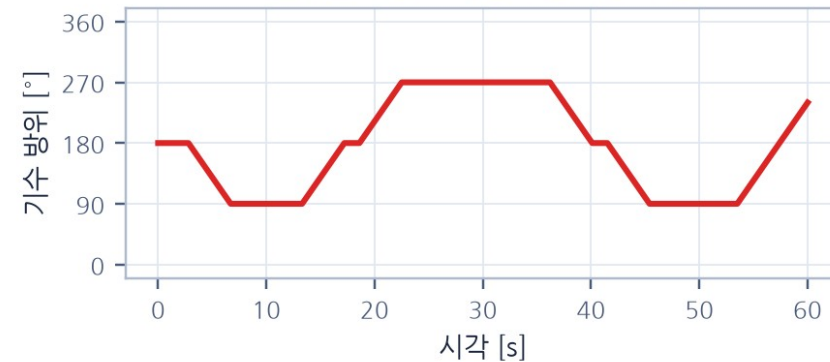
기동 시연 — 표적 2개에 기동 10줄



표적 1 대함 — 계단이 곧 기동 구간



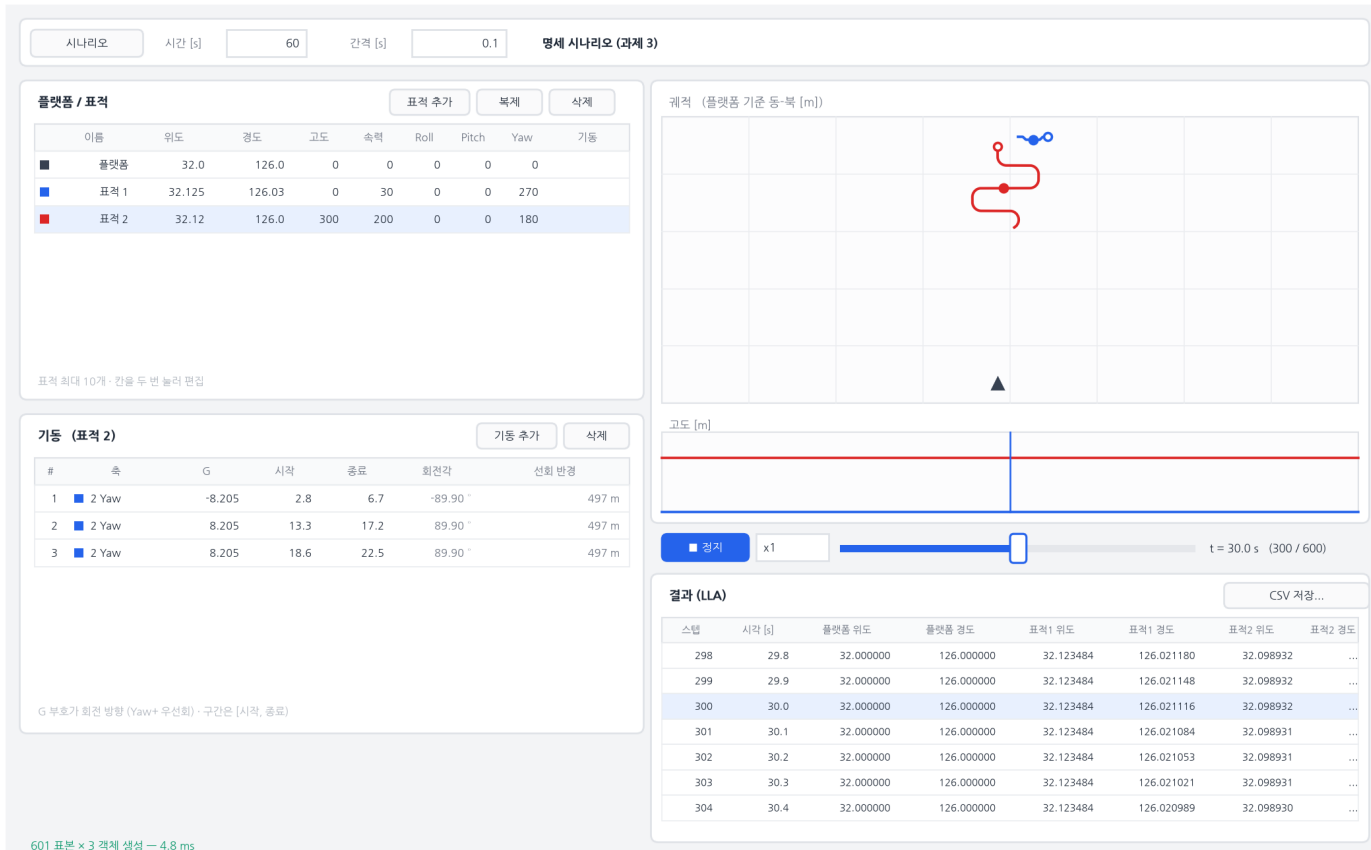
표적 2 대공 — 계단이 곧 기동 구간



03. 시뮬레이션 결과

TargetSim — 값을 바꿔 가며 볼 수 있게

표에서 숫자를 고치면 바로 다시 돌고, 그림에서 시작점과 기수를 끌 수도 있습니다.



고치면 곧바로 다시 돈다

표의 칸을 고치면 0.25 초 뒤 자동으로 다시 계산합니다. 601 표본 × 3 객체가 5 ms 안에 끝나 기다릴 일이 없습니다.

그림에서 끌어서 배치

시작점과 기수 손잡이를 마우스로 끌면 위도 · 경도 · Yaw 가 표에 그대로 반영됩니다.

결과는 표 · CSV · 그림

시각마다의 LLA 를 표로 보고, 슬라이더로 되감아 보고, CSV 로 내보냅니다. 시나리오는 .tsim 파일로 저장합니다.

03. 시뮬레이션 결과

정리

요구사항과 결과를 한 장에 놓고 보겠습니다.

과제 요구사항	이렇게 했습니다	확인
표적 10 개 · 기동 30 개	고정 배열로 상한을 두고 범위 밖은 오류 코드로 거부	완료
위치는 LLA, 속도는 V_heading	ST_TargetInit 에 그대로 . 방향은 자세각이 담당	완료
기동 = G · 축 · 시작 · 종료	ST_TargetManeuver 네 필드 . 구간 겹침도 검사	완료
ECEF 에서 상태 갱신	위치 · 속도 모두 ECEF, 출력용 LLA 를 함께 보관	완료
V_heading → Vx, Vy, Vz (추가 고민)	동체 → NED → ECEF 두 번 회전 . 속도 보존 확인	완료
제공 좌표변환 코드 사용	CoordinateTransform.c / matrixCalcLib.c 원본 그대로	완료
결과를 LLA 로 출력하고 그림으로	표 · CSV · 궤적 그림 · 고도 그림	완료

남겨 둔 것

- Pitch 가 $\pm 90^\circ$ 근처면 짐벌락 — 오일러각 대신 사원수를 쓰면 풀립니다
- 중력 · 항력 없이 기수 방향으로만 나아가는 운동학 모델입니다

다음에 이어 붙일 것

- 이 궤적에 잡음을 얹으면 곧바로 추적 필터 입력이 됩니다
- Part 2 의 좌표변환을 붙이면 안테나 기준 거리 · 방위 · 고각이 나옵니다

감사합니다

질문 주시면 답변드리겠습니다